

Cluster: AIML



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

Natural Language to OKS Query Translation Module

SUBMITTED BY

Asmit Khanal (PUL08BCT017)
Chandan Kumar Shah (PUL080BCT023)
Aditya Kumar Shah (PUL080BCT011)
Rhythm Adhikari (PUL080BCT065)

A

BE MINOR PROJECT FINAL REPORT

SUBMITTED TO THE

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN
COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
LALITPUR, NEPAL

AUGUST 26, 2026

DECLARATION OF AUTHORSHIP

This is to certify that the minor project entitled “**Natural Language to OKS Query Translation Module**” submitted for the degree of Bachelor of Engineering in Computer Engineering comprises our original work and no part of this work has been used for the award of another course or degree. We declare that this work is our own. If any part of the work is not our own, we have indicated it by acknowledging the source of that part or those part of the work with proper citations.

Name of the Student

Signature

Asmit Khanal

Chandan Kumar Shah

Aditya Kumar Shah

Rhythm Adhikari

Date: August 26, 2026

APPROVAL

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a minor project entitled “**Natural Language to OKS Query Translation Module**” submitted by Asmit Khanal (PUL08BCT017), Chandan Kumar Shah (PUL080BCT023), Aditya Kumar Shah (PUL080BCT011), and Rhythm Adhikari (PUL080BCT065) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Supervisor, Name of the Supervisor

Designation,

Department of Electronics & Computer Engineering

Pulchowk Campus, IOE, TU

Internal Examiner,

Department of Electronics & Computer Engineering

Pulchowk Campus, IOE, TU

Head of Department, Prof. Dr. Ram Krishna Maharjan

Department of Electronics & Computer Engineering,

Pulchowk Campus, IOE, TU

Date: August 26, 2026

COPYRIGHT

The authors have agreed that the library, Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering may make this project report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this report for scholarly purpose may be granted by the professor(s) who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project was done. It is understood that the recognition will be given to the author of this report and to the Department of Electronics & Computer Engineering, Pulchowk Campus, and Institute of Engineering in any use of the material of this project. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics & Computer Engineering, Pulchowk Campus, Institute of Engineering and authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head of Department

Department of Electronics & Computer Engineering

Pulchowk Campus

Pulchowk, Lalitpur

Nepal

ABSTRACT

[Content to be added...]

ACKNOWLEDGEMENTS

[Content to be added...]

TABLE OF CONTENTS

DECLARATION	i
APPROVAL	ii
COPYRIGHT	iii
ABSTRACT	iv
ACKNOWLEDGEMENTS	v
LIST OF FIGURES	viii
LIST OF TABLES	viii
ABBREVIATIONS	ix
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.3.1 Main Objective	3
1.3.2 Specific Objectives	3
1.4 Rationale	3
1.5 Scope	4
1.6 Limitations	4
1.7 Outline of the Report	4
2 LITERATURE REVIEW	5
2.1 Review of Related Theory	5
2.2 Review of Related Works	5
2.3 Gap analysis(if required)	5
3 METHODOLOGY	6
3.1 Project approach	6

3.2	System Architecture	6
3.3	System Design/Modules (UI module, backend logic, database, APIs, etc)	6
3.4	Technology Stack (languages, frameworks, tools, etc)	6
3.5	Development Methodology (agile/waterfall/iterative or suitable one) . .	6
3.6	Data Handling (input, storage, processing)	6
3.7	Testing and Validation (unit testing, integration testing, user testing) . .	6
3.8	Deployment Setup (hosting, server, environment)	6
4	IMPLEMENTATION DETAILS	7
4.1	Requirements Specification(functional/non-functional requirements, constraints)	7
4.2	System Overview (Description of implemented system, mapping methodology to implementation)	7
4.3	Development Environment(programming languages, tools, libraries and frameworks used, hardware and software specifications)	7
4.4	System Implementation(data/input handling, pre-processing steps, algo- rithm implementation, modules design and integration etc)	7
4.5	Interface and Visualization	7
4.6	Deployment and Execution(system setup, execution steps, lo- cal/server/cloud setup)	7
4.7	Challenges and Solutions(technical challenges faced, solutions found) .	7
5	RESULTS AND DISCUSSION	8
5.1	Results	8
5.2	Discussion	8
5.3	Comparative analysis(if applicable)	8
6	CONCLUSIONS AND RECOMMENDATIONS	9
6.1	Conclusions	9
6.2	Recommendations/Future Enhancements	9
	REFERENCES	9
	APPENDICES	11

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

API Application Programming Interface

LLM Large Language Model

OKS Object Kernel Support

CHAPTER 1

INTRODUCTION

1.1 Background

The ATLAS experiment at CERN’s Large Hadron Collider (LHC) relies on a highly complex data acquisition (DAQ) system to manage thousands of hardware and software components in real time. The configuration of this system, spanning detector electronics, readout chains, and computing farms, is managed by the Object Kernel Support (OKS) framework, a persistent in-memory object database built for the ATLAS Trigger and Data Acquisition (TDAQ) system [3]. OKS provides a schema-driven, object-oriented model that represents complex topologies as interconnected object graphs [3].

To meet the operational demands of successive LHC runs, ATLAS configuration management evolved from monolithic databases [2] to a Git-based service for LHC Run 3 [1]. This modern architecture provides distributed version control and full provenance tracking, ensuring every configuration state is a Git commit and the entire detector history is queryable [1].

Despite this mature infrastructure, querying the OKS database remains a specialist task. The proprietary OksQuery language uses a strict, S-expression-like syntax (e.g., (all ("InitTimeout" "2" >))) that demands detailed knowledge of class hierarchies, relationship paths, and operator semantics. Consequently, manual query formulation by shifters is error-prone, time-consuming, and inaccessible to non-specialists.

The emergence of Large Language Models (LLMs) and the Model Context Protocol (MCP) offers a new paradigm for AI-driven operations, enabling agents to interact with external databases through a standardized interface. An MCP server for the ATLAS DAQ is currently under development to allow LLMs to search the OKS database. However, a critical challenge arises: the full configuration, comprising hundreds of schema classes and thousands of objects, far exceeds any LLM’s context window. Thus, the MCP server must filter data on the backend, requiring the LLM to generate syntactically perfect OksQuery strings without seeing the complete schema.

This project addresses this challenge by developing a Natural Language to OksQuery translation module. Architected as a Retrieval-Augmented Generation (RAG) system, it

utilizes a “schema-RAG” approach where the retrieval corpus is the live, version-scoped OKS schema itself. This ensures the LLM receives only the relevant schema slice (1–3 classes) rather than the full schema, which can exceed 500 classes depending on the TDAQ release and partition, satisfying both accuracy and context-window constraints. The system is deployed securely within the CERN environment via SSH-based hosting with port tunneling, exposing the MCP server to external LLM agents without requiring them to reside inside the CERN network perimeter.

1.2 Problem Statement

The Object Kernel Support (OKS) framework is a persistent in-memory object database that provides a schema-driven, object-oriented configuration model. By supporting classes, attributes, inheritance, and composite objects, it enables the precise representation of complex hardware and software topologies. However, interacting with OKS remains a highly specialized task. Users must formulate queries using the proprietary OksQuery language or low-level C++ and Python APIs, which demands detailed, manual knowledge of class hierarchies, object states, relationship paths, and strict operator semantics.

While general-purpose Large Language Models (LLMs) present an opportunity to automate this process, they are fundamentally incapable of generating accurate OksQuery statements out-of-the-box. This failure is driven by a complete lack of parametric context regarding the specific classes, attributes, and objects within the OKS schema. Furthermore, the configuration is strictly version-specific, with schema definitions evolving across different software releases and Git commits. Because LLMs lack access to this dynamic, version-scoped context, they frequently hallucinate non-existent classes or invalid relationships, rendering their raw query accuracy insufficient for operational deployment.

To address this, we built a query translation module hosted as an MCP server via SSH port tunneling, using a version-scoped schema-RAG pipeline that injects only the relevant OKS schema slice into the LLM prompt. A deterministic validate/repair loop then checks every generated OksQuery against the authoritative schema before execution, ensuring reliable accuracy without the full configuration entering the context window.

1.3 Objectives

1.3.1 Main Objective

The main objective of this project is to develop an MCP server capable of translating natural-language questions into validated OksQuery statements through a query translation module with an integrated validate/repair loop, deployable as a tool for external LLM agents.

1.3.2 Specific Objectives

The specific objectives of this project are:

- (i) To develop a version-scoped pipeline that translates natural language into OksQuery strings by retrieving targeted OKS schema slices and enforcing syntactic correctness through a deterministic validate/repair loop.
- (ii) To deploy the translation module as a Model Context Protocol (MCP) server within the CERN compute infrastructure, utilizing SSH port tunneling to securely expose the live TDAQ configuration to external LLM agents.
- (iii) To evaluate translation accuracy by executing generated queries against the native C++ backend and analyzing performance metrics such as valid-syntax rate, result-set equivalence, and execution latency across stratified query difficulties.

1.4 Rationale

The ATLAS Data Acquisition (DAQ) system relies on the Object Kernel Support (OKS) framework, a complex, version-controlled object database. Querying this configuration during time-critical operations requires specialized knowledge of the proprietary OksQuery language, creating a significant bottleneck for control room operators.

While Large Language Models (LLMs) could automate this querying process through natural language, raw models fail because they hallucinate syntax for undocumented, proprietary languages and cannot ingest massive, constantly evolving schemas within their context limits.

This project addresses these fundamental limitations by developing a retrieval-augmented translation module deployed as a Model Context Protocol (MCP) server. By utilizing a version-scoped Schema-RAG pipeline, the system feeds the LLM only the

exact schema slice it needs. Furthermore, by forcing the LLM to output an Abstract Syntax Tree (AST) that undergoes a strict validate-and-repair loop against the authoritative, version-bound OKS schema, the system eliminates syntax hallucinations before execution. Ultimately, this establishes a robust and verifiable paradigm for safely deploying generative AI within high-stakes scientific infrastructure.

1.5 Scope

The system is scoped to translating natural-language questions about the ATLAS DAQ configuration into validated OksQuery statements for read-only retrieval, processing one query at a time within a single, explicitly resolved configuration version (TDAQ release, Git revision, or run number), with schema retrieval performed dynamically per request using a version-scoped lexical pipeline (exact match, alias lookup, BM25) keyed by schema fingerprint, and the retrieval index regenerated from the OKS schema whenever the configuration changes. The module generates an intermediate AST representation rather than raw OksQuery strings, validated deterministically against the version-bound OKS schema before compilation, and is deployed as an MCP server hosted within CERN compute infrastructure and exposed to external LLM agents through SSH port tunneling, with conversational persistence and session state managed by the consuming LLM agent rather than the translation module. The system does not write, modify, or delete configuration data, does not provide operational recommendations or interpret query results beyond factual reporting, does not access live running partition memory or process telemetry data, does not handle user authentication (security relies on the CERN SSH tunnel and institutional network perimeter), and does not support non-English natural-language input or cross-version comparison queries.

1.6 Limitations

[Content to be added...]

1.7 Outline of the Report

[Content to be added...]

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Related Theory

[Content to be added...]

2.2 Review of Related Works

[Content to be added...]

2.3 Gap analysis(if required)

[Content to be added...]

CHAPTER 3

METHODOLOGY

3.1 Project approach

[Content to be added...]

3.2 System Architecture

[Content to be added...]

3.3 System Design/Modules (UI module, backend logic, database, APIs, etc)

[Content to be added...]

3.4 Technology Stack (languages, frameworks, tools, etc)

[Content to be added...]

3.5 Development Methodology (agile/waterfall/iterative or suitable one)

[Content to be added...]

3.6 Data Handling (input, storage, processing)

[Content to be added...]

3.7 Testing and Validation (unit testing, integration testing, user testing)

[Content to be added...]

3.8 Deployment Setup (hosting, server, environment)

[Content to be added...]

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Requirements Specification(functional/non-functional requirements, constraints)

[Content to be added...]

4.2 System Overview (Description of implemented system, mapping methodology to implementation)

[Content to be added...]

4.3 Development Environment(programming languages, tools, libraries and frameworks used, hardware and software specifications)

[Content to be added...]

4.4 System Implementation(data/input handling, pre-processing steps, algorithm implementation, modules design and integration etc)

[Content to be added...]

4.5 Interface and Visualization

[Content to be added...]

4.6 Deployment and Execution(system setup, execution steps, local/server/cloud setup)

[Content to be added...]

4.7 Challenges and Solutions(technical challenges faced, solutions found)

[Content to be added...]

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Results

[Content to be added...]

5.2 Discussion

[Content to be added...]

5.3 Comparative analysis(if applicable)

[Content to be added...]

CHAPTER 6

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusions

[Content to be added...]

6.2 Recommendations/Future Enhancements

[Content to be added...]

REFERENCES

- [1] ATLAS Collaboration, “The git based ATLAS data acquisition configuration service in LHC Run 3,” *Journal of Physics: Conference Series*.
- [2] E. Alexandrov *et al.*, “The ATLAS DAQ system online configurations database service challenge,” *Journal of Physics: Conference Series*, vol. 119, p. 022004, 2007.
- [3] “The OKS persistent in-memory object manager,” ATLAS TDAQ Technical Paper.
- [4] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, “Knowledge Graph-Guided Retrieval Augmented Generation,” arXiv:2502.06864, NAACL, 2025.

APPENDICES

[Content to be added...]